

HOMEWORK 4

Kavya Saranya K V
2025115152

Q1. Implement a stack using queue data structure.

Code:

```
#include <stdio.h>
#define MAX 100

typedef struct {
    int arr[MAX];
    int front, rear;
} Queue;

void enqueue(Queue *q, int val) {
    q->arr[q->rear++] = val;
}

int dequeue(Queue *q) {
    return q->arr[q->front++];
}

int isEmpty(Queue *q) {
    return q->front == q->rear;
}

void push(Queue *q, int val) {
    int size = q->rear - q->front;
    enqueue(q, val);
    for (int i = 0; i < size; i++) {
        enqueue(q, dequeue(q));
    }
}

int pop(Queue *q) {
    return dequeue(q);
}

int main() {
    Queue q;
    q.front = q.rear = 0;

    push(&q, 10);
    push(&q, 20);
    push(&q, 30);

    printf("Stack (using queue) pop order: ");
    while (!isEmpty(&q))
        printf("%d ", pop(&q));
    printf("\n");

    return 0;
}
```

Sample Output:

Stack (using queue) pop order: 30 20 10

Q2. Implement a queue using stack data structure.

Code:

```

#include <stdio.h>
#define MAX 100

typedef struct {
    int arr[MAX];
    int top;
} Stack;

void push(Stack *s, int val) {
    s->arr[++(s->top)] = val;
}

int pop(Stack *s) {
    return s->arr[(s->top)--];
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

typedef struct {
    Stack in, out;
} Queue;

void enqueue(Queue *q, int val) {
    push(&q->in, val);
}

int dequeue(Queue *q) {
    if (isEmpty(&q->out)) {
        while (!isEmpty(&q->in))
            push(&q->out, pop(&q->in));
    }
    return pop(&q->out);
}

int isEmptyQueue(Queue *q) {
    return isEmpty(&q->in) && isEmpty(&q->out);
}

int main() {
    Queue q;
    q.in.top = q.out.top = -1;

    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);

    printf("Queue (using stack) dequeue order: ");
    while (!isEmptyQueue(&q))
        printf("%d ", dequeue(&q));
    printf("\n");

    return 0;
}

```

Sample Output:

Queue (using stack) dequeue order: 10 20 30

Q3. Write a C program using stack data structure to implement managing the history of web pages visited.

Code:

```

#include <stdio.h>

```

```

#include <string.h>
#define MAX 100

typedef struct {
    char arr[MAX][50];
    int top;
} Stack;

void push(Stack *s, char *page) {
    strcpy(s->arr[++(s->top)], page);
}

void pop(Stack *s, char *dest) {
    strcpy(dest, s->arr[(s->top)--]);
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

typedef struct {
    Stack back, forward;
    char current[50];
} Browser;

void visit(Browser *b, char *page) {
    push(&b->back, b->current);
    strcpy(b->current, page);
    b->forward.top = -1;
}

void goBack(Browser *b) {
    if (isEmpty(&b->back)) {
        printf("No previous page\n");
        return;
    }
    push(&b->forward, b->current);
    pop(&b->back, b->current);
}

void goForward(Browser *b) {
    if (isEmpty(&b->forward)) {
        printf("No forward page\n");
        return;
    }
    push(&b->back, b->current);
    pop(&b->forward, b->current);
}

int main() {
    Browser b;
    b.back.top = b.forward.top = -1;
    strcpy(b.current, "home.com");

    visit(&b, "google.com");
    visit(&b, "github.com");
    visit(&b, "leetcode.com");

    printf("Current page: %s\n", b.current);

    goBack(&b);
    printf("After back: %s\n", b.current);

    goBack(&b);
}

```

```

        printf("After back: %s\n", b.current);

        goForward(&b);
        printf("After forward: %s\n", b.current);

        return 0;
    }

```

Sample Output:

```

Current page: leetcode.com
After back: github.com
After back: google.com
After forward: github.com

```

Q4. Write a C program using a temporary stack to sort a stack.

Code:

```

#include <stdio.h>
#define MAX 100

typedef struct {
    int arr[MAX];
    int top;
} Stack;

void push(Stack *s, int val) {
    s->arr[++(s->top)] = val;
}

int pop(Stack *s) {
    return s->arr[(s->top)--];
}

int peek(Stack *s) {
    return s->arr[s->top];
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

void sortStack(Stack *input) {
    Stack temp;
    temp.top = -1;

    while (!isEmpty(input)) {
        int val = pop(input);
        while (!isEmpty(&temp) && peek(&temp) > val) {
            push(input, pop(&temp));
        }
        push(&temp, val);
    }

    while (!isEmpty(&temp)) {
        push(input, pop(&temp));
    }
}

int main() {
    Stack s;
    s.top = -1;

    push(&s, 34);
    push(&s, 3);

```

```

    push(&s, 31);
    push(&s, 98);
    push(&s, 92);
    push(&s, 23);

    sortStack(&s);

    printf("Stack popped in ascending order: ");
    while (!isEmpty(&s))
        printf("%d ", pop(&s));
    printf("\n");

    return 0;
}

```

Sample Output:

Stack popped in ascending order: 3 23 31 34 92 98

Q5. The stock span problem: given an array of daily stock prices, find the span of the stock price for each day (the maximum number of consecutive days up to and including the current day where the price was less than or equal to the current day's price).

Code:

```

#include <stdio.h>
#define MAX 100

typedef struct {
    int arr[MAX];
    int top;
} Stack;

void push(Stack *s, int val) {
    s->arr[++(s->top)] = val;
}

int pop(Stack *s) {
    return s->arr[(s->top)--];
}

int peek(Stack *s) {
    return s->arr[s->top];
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

void stockSpan(int price[], int span[], int n) {
    Stack s;
    s.top = -1;

    for (int i = 0; i < n; i++) {
        while (!isEmpty(&s) && price[peek(&s)] <= price[i])
            pop(&s);

        span[i] = isEmpty(&s) ? (i + 1) : (i - peek(&s));
        push(&s, i);
    }
}

int main() {
    int price[] = {100, 80, 60, 70, 60, 75, 85};
    int n = 7;
    int span[MAX];
}

```

```

        stockSpan(price, span, n);

        printf("Stock span: ");
        for (int i = 0; i < n; i++)
            printf("%d ", span[i]);
        printf("\n");

        return 0;
    }

```

Sample Output:

Stock span: 1 1 1 2 1 4 6

Q6. Given an array, for each element find the nearest element to its right that has a higher frequency than the current element. If no such element exists, set the value to -1. Implement using stack.

Code:

```

#include <stdio.h>
#define MAX 100

typedef struct {
    int arr[MAX];
    int top;
} Stack;

void push(Stack *s, int val) {
    s->arr[++(s->top)] = val;
}

int pop(Stack *s) {
    return s->arr[(s->top)--];
}

int peek(Stack *s) {
    return s->arr[s->top];
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

int main() {
    int arr[] = {2, 1, 1, 3, 2, 1};
    int n = 6;
    int freq[MAX], result[MAX];
    int i, j;

    for (i = 0; i < n; i++) {
        freq[i] = 0;
        for (j = 0; j < n; j++)
            if (arr[j] == arr[i])
                freq[i]++;
    }

    Stack s;
    s.top = -1;

    for (i = n - 1; i >= 0; i--) {
        while (!isEmpty(&s) && freq[peek(&s)] <= freq[i])
            pop(&s);

        result[i] = isEmpty(&s) ? -1 : arr[peek(&s)];
        push(&s, i);
    }
}

```

```

    }

    printf("Next greater frequency element: ");
    for (i = 0; i < n; i++)
        printf("%d ", result[i]);
    printf("\n");

    return 0;
}

```

Sample Output:

Next greater frequency element: 1 -1 -1 2 1 -1

Q7. Given a pattern containing only 'I' (increasing) and 'D' (decreasing), print the smallest number following that pattern using digits 1-9 with no repeats, using a stack.

Code:

```

#include <stdio.h>
#include <string.h>
#define MAX 100

typedef struct {
    int arr[MAX];
    int top;
} Stack;

void push(Stack *s, int val) {
    s->arr[++(s->top)] = val;
}

int pop(Stack *s) {
    return s->arr[(s->top)--];
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

void printSmallest(char *pattern) {
    int n = strlen(pattern);
    Stack s;
    s.top = -1;

    for (int i = 0; i <= n; i++) {
        push(&s, i + 1);

        if (i == n || pattern[i] == 'I') {
            while (!isEmpty(&s)) {
                printf("%d", pop(&s));
            }
        }
        printf("\n");
    }
}

int main() {
    printSmallest("D");
    printSmallest("DIDI");
    return 0;
}

```

Sample Output:

21
21435

Q8. Given an array of digits (0-9), find the minimum possible sum of two numbers formed using all the digits, using a queue.

Code:

```
#include <stdio.h>
#define MAX 100

typedef struct {
    int arr[MAX];
    int front, rear;
} Queue;

void enqueue(Queue *q, int val) {
    q->arr[q->rear++] = val;
}

int dequeue(Queue *q) {
    return q->arr[q->front++];
}

int isEmpty(Queue *q) {
    return q->front == q->rear;
}

void sortArray(int arr[], int n) {
    int i, j, minIdx, temp;
    for (i = 0; i < n - 1; i++) {
        minIdx = i;
        for (j = i + 1; j < n; j++)
            if (arr[j] < arr[minIdx])
                minIdx = j;
        temp = arr[i];
        arr[i] = arr[minIdx];
        arr[minIdx] = temp;
    }
}

int main() {
    int digits[] = {6, 8, 4, 5, 2, 3};
    int n = 6;

    sortArray(digits, n);

    Queue q;
    q.front = q.rear = 0;
    for (int i = 0; i < n; i++)
        enqueue(&q, digits[i]);

    int num1 = 0, num2 = 0;
    int turn = 0;
    while (!isEmpty(&q)) {
        if (turn == 0)
            num1 = num1 * 10 + dequeue(&q);
        else
            num2 = num2 * 10 + dequeue(&q);
        turn = 1 - turn;
    }

    printf("Minimum sum = %d (numbers %d and %d)\n", num1 + num2, num1, num2);
    return 0;
}
```

Sample Output:

Minimum sum = 604 (numbers 246 and 358)

Q9. Given an array and a positive integer k, find the first negative integer for each window of size k. If a window has no negative integer, print 0 for that window. Implement using a queue.

Code:

```
#include <stdio.h>
#define MAX 100

typedef struct {
    int arr[MAX];
    int front, rear;
} Queue;

void enqueue(Queue *q, int val) {
    q->arr[q->rear++] = val;
}

int dequeue(Queue *q) {
    return q->arr[q->front++];
}

int frontVal(Queue *q) {
    return q->arr[q->front];
}

int isEmpty(Queue *q) {
    return q->front == q->rear;
}

int main() {
    int arr[] = {-8, 2, 3, -6, 1};
    int n = 5, k = 2;
    int result[MAX], ri = 0;
    int i;

    Queue q;
    q.front = q.rear = 0;

    for (i = 0; i < k; i++)
        if (arr[i] < 0)
            enqueue(&q, i);

    for (i = k; i <= n; i++) {
        if (isEmpty(&q))
            result[ri++] = 0;
        else
            result[ri++] = arr[frontVal(&q)];

        if (i == n)
            break;

        while (!isEmpty(&q) && frontVal(&q) <= i - k)
            dequeue(&q);

        if (arr[i] < 0)
            enqueue(&q, i);
    }

    printf("First negative in each window: ");
    for (i = 0; i < ri; i++)
        printf("%d ", result[i]);
    printf("\n");

    return 0;
}
```

Sample Output:

First negative in each window: -8 0 -6 -6

Q10. Given a matrix where each cell can be 0 (empty), 1 (fresh orange), or 2 (rotten orange), find the minimum time for all fresh oranges to rot, given that a rotten orange rots its up/down/left/right neighbours each minute. Return -1 if not all oranges can rot. Implement using a queue.

Code:

```
#include <stdio.h>
#define MAX 100

typedef struct {
    int r, c;
} Cell;

typedef struct {
    Cell arr[MAX];
    int front, rear;
} Queue;

void enqueue(Queue *q, Cell val) {
    q->arr[q->rear++] = val;
}

Cell dequeue(Queue *q) {
    return q->arr[q->front++];
}

int isEmpty(Queue *q) {
    return q->front == q->rear;
}

int main() {
    int grid[3][5] = {
        {2, 1, 0, 2, 1},
        {1, 0, 1, 2, 1},
        {1, 0, 0, 2, 1}
    };
    int m = 3, n = 5;
    int i, j;
    int freshCount = 0;

    Queue q;
    q.front = q.rear = 0;

    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) {
            if (grid[i][j] == 2) {
                Cell c = {i, j};
                enqueue(&q, c);
            } else if (grid[i][j] == 1) {
                freshCount++;
            }
        }
    }

    int minutes = 0;
    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    while (!isEmpty(&q) && freshCount > 0) {
        int size = q.rear - q.front;
```

```

int k;
for (k = 0; k < size; k++) {
    Cell cur = dequeue(&q);
    int d;
    for (d = 0; d < 4; d++) {
        int nr = cur.r + dr[d];
        int nc = cur.c + dc[d];
        if (nr >= 0 && nr < m && nc >= 0 && nc < n && grid[nr][nc] == 1) {
            grid[nr][nc] = 2;
            freshCount--;
            Cell newCell = {nr, nc};
            enqueue(&q, newCell);
        }
    }
    minutes++;
}

if (freshCount > 0)
    printf("Minimum time = -1\n");
else
    printf("Minimum time = %d\n", minutes);

return 0;
}

```

Sample Output:

Minimum time = 2